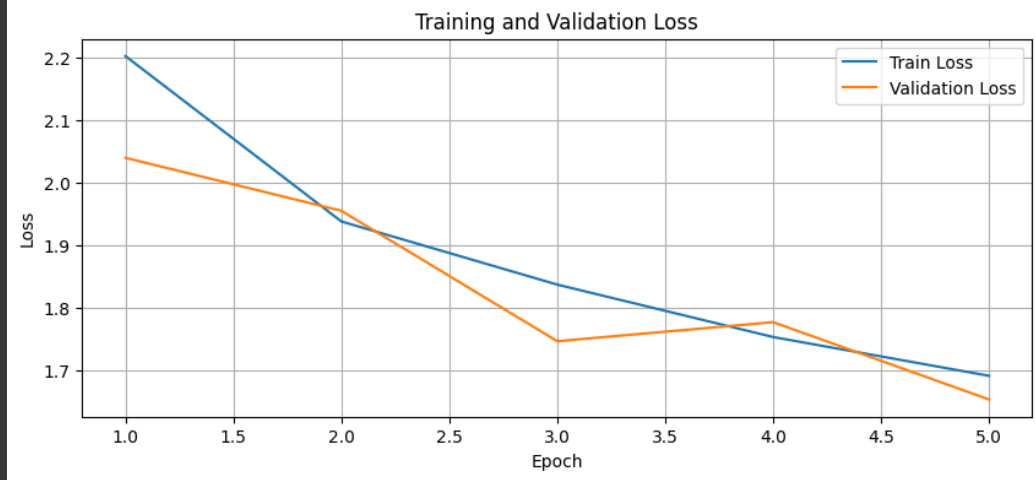
1. **Record Baseline Performance and the concepts about overfitting**

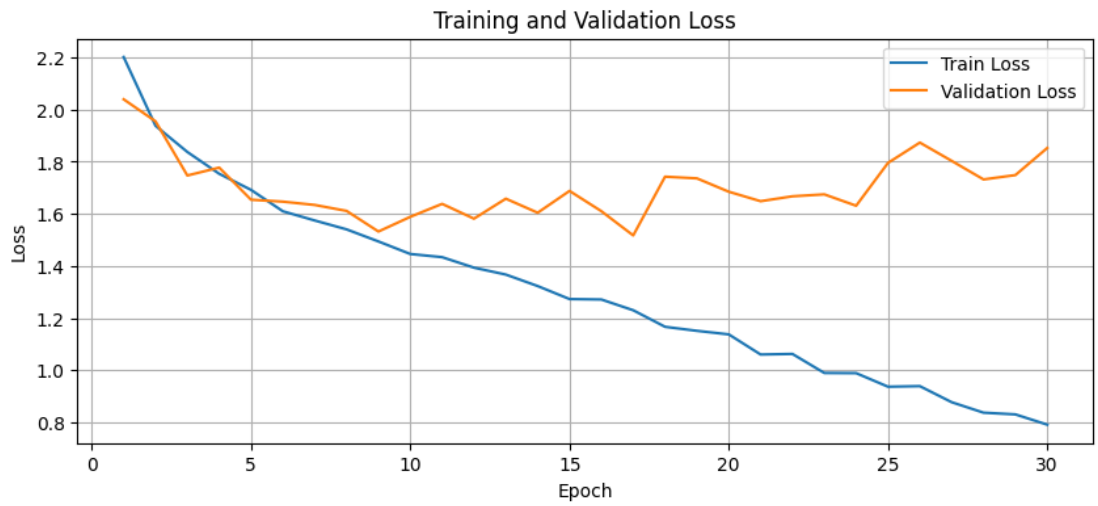
   1. Run the provided example and first train for 5 epochs, recording the training and validation loss/accuracy

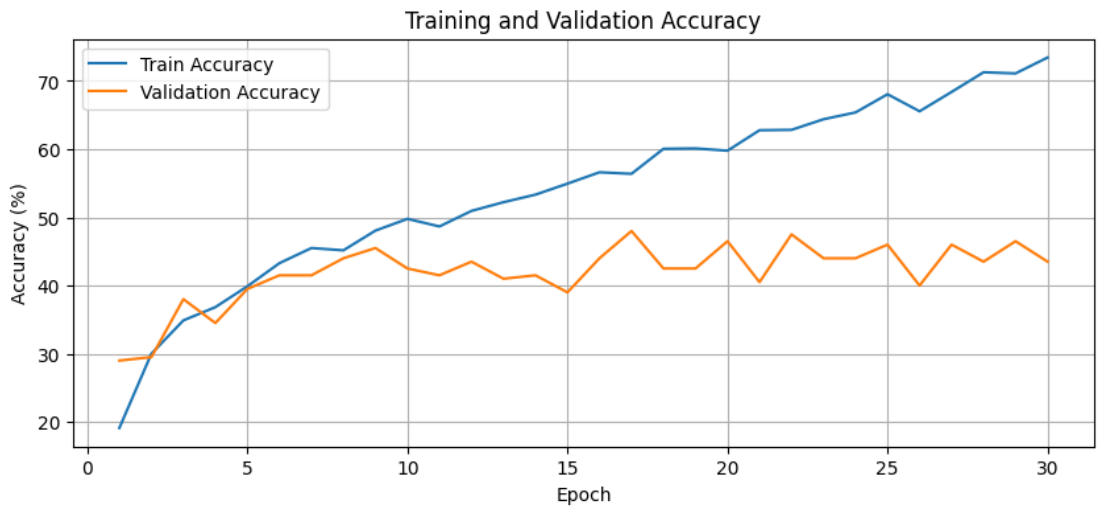
```
100%|                                        5/5 [00:05<00:00,  1.02s/it]

Epoch: 01 | Epoch Time: 0m 1s
    Train Loss: 2.202 | Train Acc: 19.11%
     Val. Loss: 2.040 |  Val. Acc: 29.00%

Epoch: 02 | Epoch Time: 0m 1s
    Train Loss: 1.938 | Train Acc: 30.00%
     Val. Loss: 1.955 |  Val. Acc: 29.50%

Epoch: 03 | Epoch Time: 0m 0s
    Train Loss: 1.838 | Train Acc: 34.89%
     Val. Loss: 1.747 |  Val. Acc: 38.00%

Epoch: 04 | Epoch Time: 0m 0s
    Train Loss: 1.754 | Train Acc: 36.83%
     Val. Loss: 1.778 |  Val. Acc: 34.50%

Epoch: 05 | Epoch Time: 0m 0s
    Train Loss: 1.692 | Train Acc: 39.89%
     Val. Loss: 1.654 |  Val. Acc: 39.50%
```



   2. Then extend the training to 30 epochs, re-run the training **from scratch**, and again record the loss and accuracy

```
100%|                                        30/30 [00:24<00:00,  1.23it/s]
Epoch: 01 | Epoch Time: 0m 0s
    Train Loss: 2.202 | Train Acc: 19.11%
     Val. Loss: 2.040 |  Val. Acc: 29.00%
Epoch: 02 | Epoch Time: 0m 0s
    Train Loss: 1.938 | Train Acc: 30.00%
     Val. Loss: 1.955 |  Val. Acc: 29.50%
Epoch: 03 | Epoch Time: 0m 0s
    Train Loss: 1.838 | Train Acc: 34.89%
     Val. Loss: 1.747 |  Val. Acc: 38.00%
Epoch: 04 | Epoch Time: 0m 0s
    Train Loss: 1.754 | Train Acc: 36.83%
     Val. Loss: 1.778 |  Val. Acc: 34.50%
```





3. What is **overfitting and underfitting** in deep learning? Please describe it in your own words

   Overfitting happens when a model learns the training data too well (training acc >>> validating acc), Underfitting happens when a model is too simple to capture the underlying patterns in the data (training acc and validating acc both are bad and there's no gap).

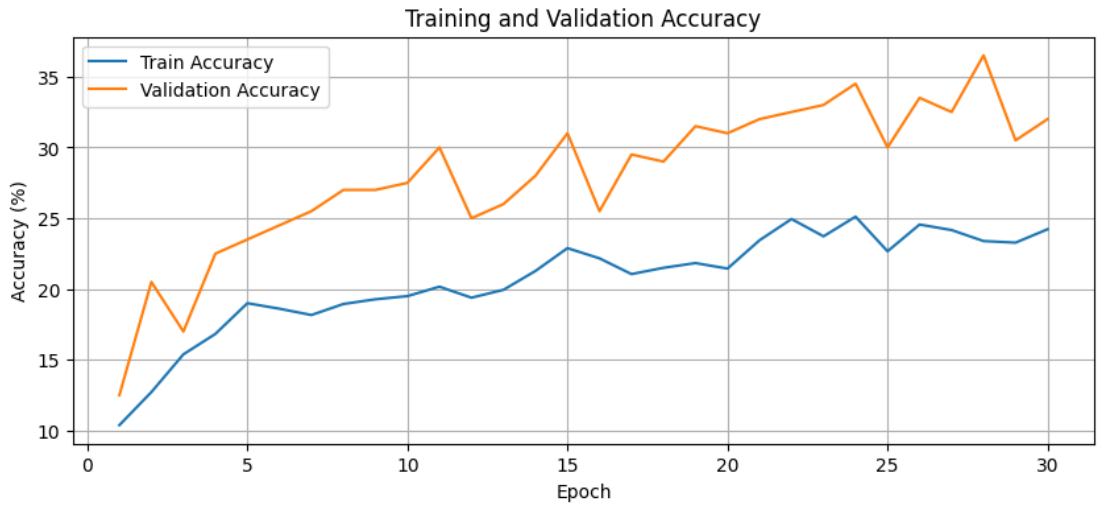
4. Compare the graphs from epoch = 5 and epoch = 30. Do these results indicate underfitting or overfitting? Explain why
   Overfitting at "epoch = 30", because validating acc and validating loss stop improving at 10+ epoch.
   Underfitting at "epoch = 5", because validating acc and validating loss are both low and there's no gap between these 2 lines.

2. Add Dropout

1. What is Dropout? Please briefly explain what dropout does in a neural network in your own words

   Dropout is a regularization technique to prevent overfitting. It temporarily removes a fraction of neurons in a layer on each forward pass.

2. Add a **Dropout layer with p = 0.5** after the hidden layer, and then train the model again for **30 epochs**, record the new **loss and accuracy**



3. Did dropout help reduce or fix the overfitting issue? Why do you think so?

   Yes, without dropout, the training acc went to 70% which differs largely from the validating one (40%). However, after adding dropout, although

validating acc decreases, it no longer holds a big difference against training acc (32% vs 25%). Evaluating the loss will have the same result.

3. Add cross-validation

1. Implement K-fold stratified cross-validation for K = 5 on your existing small_trainset in Cross-Validation code block and then report: a table of per-fold validation loss/accuracy, plus mean ± std across folds.

```
========== CV Summary ==========
Fold 1: best Val Loss 1.876, Val Acc 35.75%
Fold 2: best Val Loss 1.707, Val Acc 41.00%
Fold 3: best Val Loss 1.781, Val Acc 37.75%
Fold 4: best Val Loss 1.779, Val Acc 34.75%
Fold 5: best Val Loss 1.727, Val Acc 41.00%

[CV Aggregate] Val Loss mean 1.774 | Val Acc mean 38.05% ± 2.60%
```

Mean Validation Loss: 1.774

Mean Validation Accuracy: 38.05% ± 2.60%

2. Choose training length: take the median of best epochs across folds, and retrain once on the full training subset for that CV-suggested number of epochs and evaluate on the (unchanged) test set

```
[Final Retrain] Training fresh MLP on FULL small_trainset for 5 epochs...
Epoch 01 | Train Loss 2.217 Acc 17.00% | Test  Loss 1.971 Acc 31.20%
Epoch 02 | Train Loss 1.934 Acc 30.50% | Test  Loss 1.917 Acc 30.20%
Epoch 03 | Train Loss 1.800 Acc 36.60% | Test  Loss 1.760 Acc 37.70%
Epoch 04 | Train Loss 1.690 Acc 40.30% | Test  Loss 1.773 Acc 36.70%
Epoch 05 | Train Loss 1.628 Acc 41.65% | Test  Loss 1.711 Acc 39.60%

[Final Result] (best @ epoch 05) Test Loss 1.711 | Test Acc 39.60%
```

3. How different are the per-fold results? What does the std tell you about variance/instability of training on a tiny dataset?

Validation Loss Range: 1.707 to 1.876, Validation Accuracy Range: 34.75% to 41.00%. I would say it's quite different considering how low the accuracy is (only 38.05%).

This standard deviation (2.6%) of training with a tiny dataset is relatively high for a cross-validation result. This means it's instable, usually

cross-validation won't have such wide range of difference.

4. Add ensemble learning

1. Build an ensemble using the best checkpoint from each fold (optionally including your final retrained model), and at test time average their softmax probabilities across models, then predict via the argmax of the averaged distribution, and then evaluate the ensemble on the same fixed test set as before

```
[Ensemble] Found 6 checkpoints:
 - mlp_cv_fold1.pt
 - mlp_cv_fold2.pt
 - mlp_cv_fold3.pt
 - mlp_cv_fold4.pt
 - mlp_cv_fold5.pt
 - mlp_full_retrain.pt

========== Ensemble on Test Set ==========
Models ensembled: 6
Test Loss (ensemble): 1.634
Test Acc  (ensemble): 44.60%
```

2. Compare three results side-by-side by including a concise table showing their Test Accuracy and plus difference Δ vs Baseline, did the ensemble improve test accuracy compared to the best single model?

```
================ Test Accuracy (No Recompute) ================
Baseline (no CV)      | Test Acc: 40.40%
CV-guided Final       | Test Acc: 39.60%
Ensemble              | Test Acc: 44.60%
Δ vs Baseline → CV-guided Final: -0.80%
Δ vs Baseline → Ensemble       :  4.20%
==============================================================
```

3. Why can averaging across folds reduce variance and produce more stable predictions on small data? Briefly discuss trade-offs (extra storage/compute, diminishing returns)

   In short, averaging across folds reduces variance and improves stability. Additionally, we don't need too much time to compute and too

much memory to store records because the data number is relatively small.

Of course, it's not that it's better to have more folds as long as we're modeling on small data. Going from 1→5 folds give substantial gains, however, beyond 10-20 folds, gains become marginal while costs scale linearly.